

SWE300011 - IoT Programming

IoT Individual Practical

Report

IoT-based Smart Medication Storage Box

Rye Mohammed Fahim

104557267

Table of Contents

Table of Contents	2
1 - Topic Background	3
2 - Proposed System	4
3 - Conceptual Design	5
3.1 Block Diagrams	5
3.1.1 Physical Layer	5
3.1.2 Software Layer	6
3.2 UML Diagrams	7
3.2.1 Use Case Diagram	7
3.2.2 Activity Diagram	8
4 – Implementation	9
4.1 Sensors:	9
4.1.1 Analog	9
4.1.2 Digital	9
4.2 Actuators	10
4.3 Input Devices	10
4.4 Software/Libraries	11
4.4. 1 Arduino Libraries	11
4.4.2 Python Libraries-	11
4.4.3 System Software -	11
Resources	12

1 - Topic Background

The secure storage of medicine is crucial for ensuring the safety of individuals. Traditional medication storage boxes typically only provide basic physical storage and do not include advanced security or environment measuring features. This creates a significant gap in medication management for individuals who require long-term or temperature sensitive medicines. As home-based treatment and self-management of healthcare becomes more accessible, there is a growing need for the secure storage of medication while continuously monitoring conditions of storage.

Many medications such as insulin, vaccinations, and other temperature sensitive drugs require certain storage conditions to maintain their effectiveness and safety. When exposed to unsuitable temperatures and humidity, the medication can degrade over time and lead to serious health risks or reduced effectiveness. Additionally, many of these medications are prescription only and require secure storage to prevent misuse, theft, or accidental ingestion by children or vulnerable individuals. Traditional home storage solutions do not provide enough protection against these risks and do not alert the owners when an unsafe condition is occurring. The proposed solution aims to improve medical storage safety and provide greater protection and control to users.

2 - Proposed System

To address these challenges, the system that has been proposed is an IoT-based Smart Medication Storage Box, designed to provide secure storage, environmental monitoring, and real-time alerts. The system consists of physical hardware, edge computing, database storage, and a webpage. The primary goal of the system is to improve the way medicine is stored. The system is secured with a lock, keypad, and intrusion alarms. The system also actively monitors the temperature and humidity inside of the box, triggering alarms if either state reaches a critical level.

The physical layer uses an Arduino Uno as a microcontroller which is responsible for controlling hardware components for the box. A keypad is used for allowing authorised access for unlocking and locking the box. A servo motor functions as the lock, physically securing the lid closed. To detect unauthorised access, a Light Dependant Resistor (LDR) sensor is used to monitor light, which detects when the box is opened without the password and is responsible for triggering a break-in alarm until the password is entered. A DHT11 sensor is used to continuously monitor the temperature and humidity levels inside of the box. If the environmental conditions inside the box reach an unsafe zone, the sensor triggers a temperature or humidity alarm that stops when the condition is back to normal. The alarm consists of a buzzer and RGB LED that uses different frequencies and colours for different alarms. There is also a dedicated mute button to temporarily silence the buzzer while the alarm is going off while maintaining the alarmed state.

The edge layer uses an edge server that communicates back and forth with the Arduino using serial communication. The edge server is responsible for processing incoming data from the system and storing it in a MySQL database. The server also executes edge logic to control the operations of the box, such as triggering alarms. The edge server receives sensor data from the Arduino and uses conditional logic and rules to determine how the system should operate.

A Flask-based web application is hosted on the edge server to provide real-time updates to a dashboard that displays live sensor readings, real-time alerts, box lock state and temperature analytics. The webpage also has a data page that allows users to view all sensor, alarm and access records uploaded to the database. There is also a control page that has a lock and unlock button which allows the user to remotely lock and unlock the box. The webpage allows for the user to monitor, receive alerts, and control the box remotely.

3 - Conceptual Design

3.1 Block Diagrams

3.1.1 Physical Layer

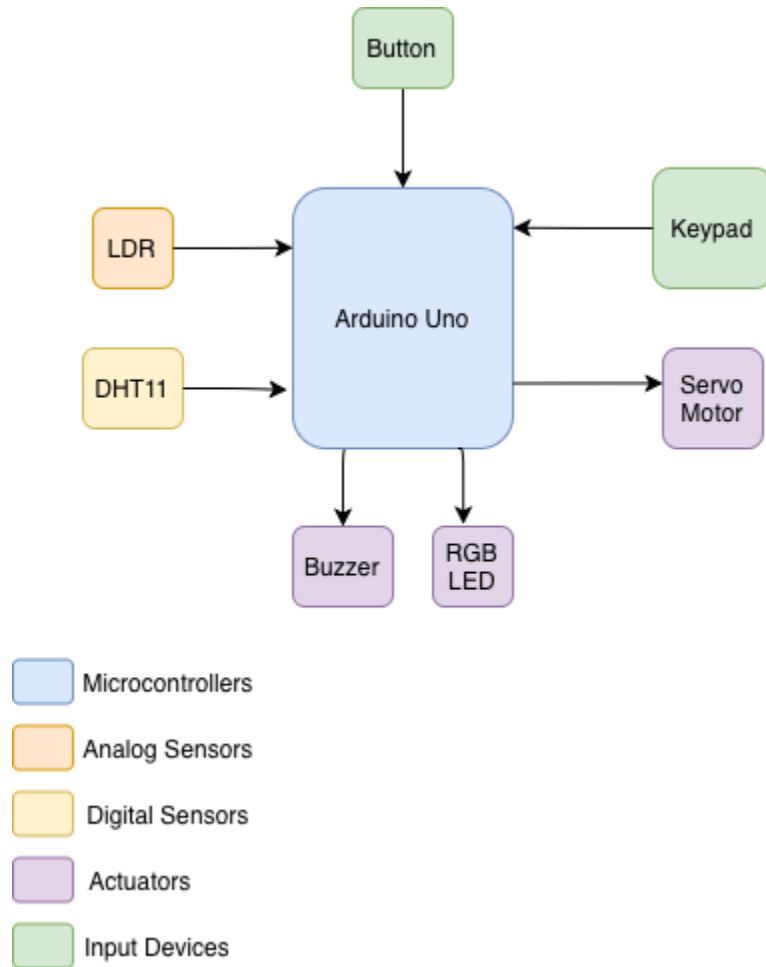


Figure 1. Physical Layer Block Diagram

This diagram illustrates the hardware components connected to the Arduino Uno. It shows the input devices (Button, Keypad), and the sensors (LDR, DHT11) providing data to the Arduino, while the actuators (Buzzer, RGB LED) get controlled by the Arduino.

3.1.2 Software Layer

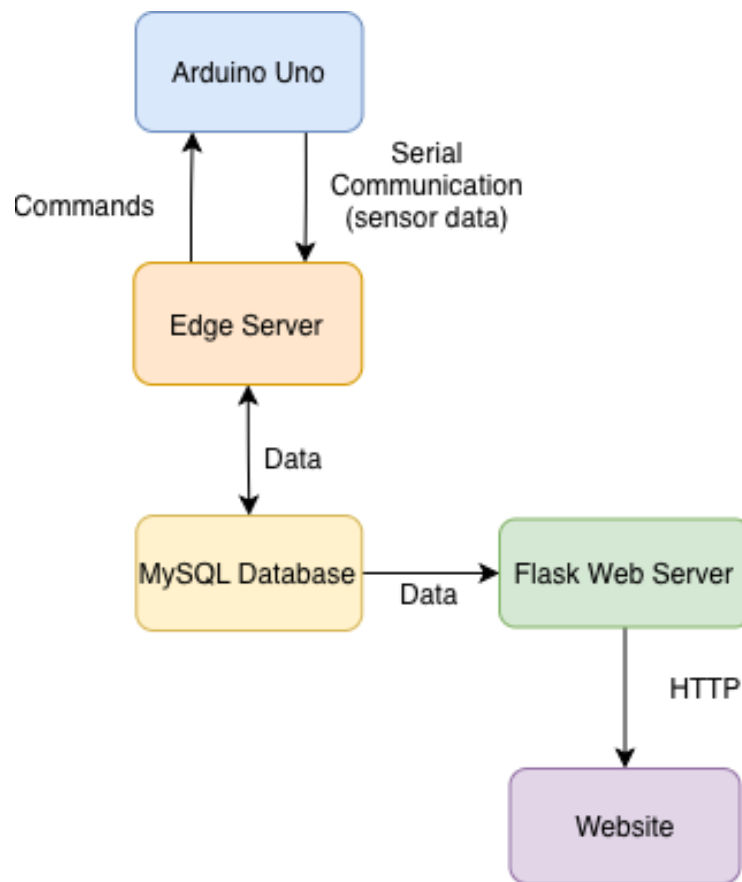


Figure 2. Software Layer Block Diagram

This diagram illustrates the flow of data through the physical layer, edge layer, database, and user interface components of the system. The Arduino Uno communicates with the Edge Server via serial communication by sending the sensor data and receiving commands from the Edge Server. The Edge Server processes the received data and stores it into the MySQL database. The Flask Web Server reads the database and serves the web interface to the user via HTTP.

3.2 UML Diagrams

3.2.1 Use Case Diagram

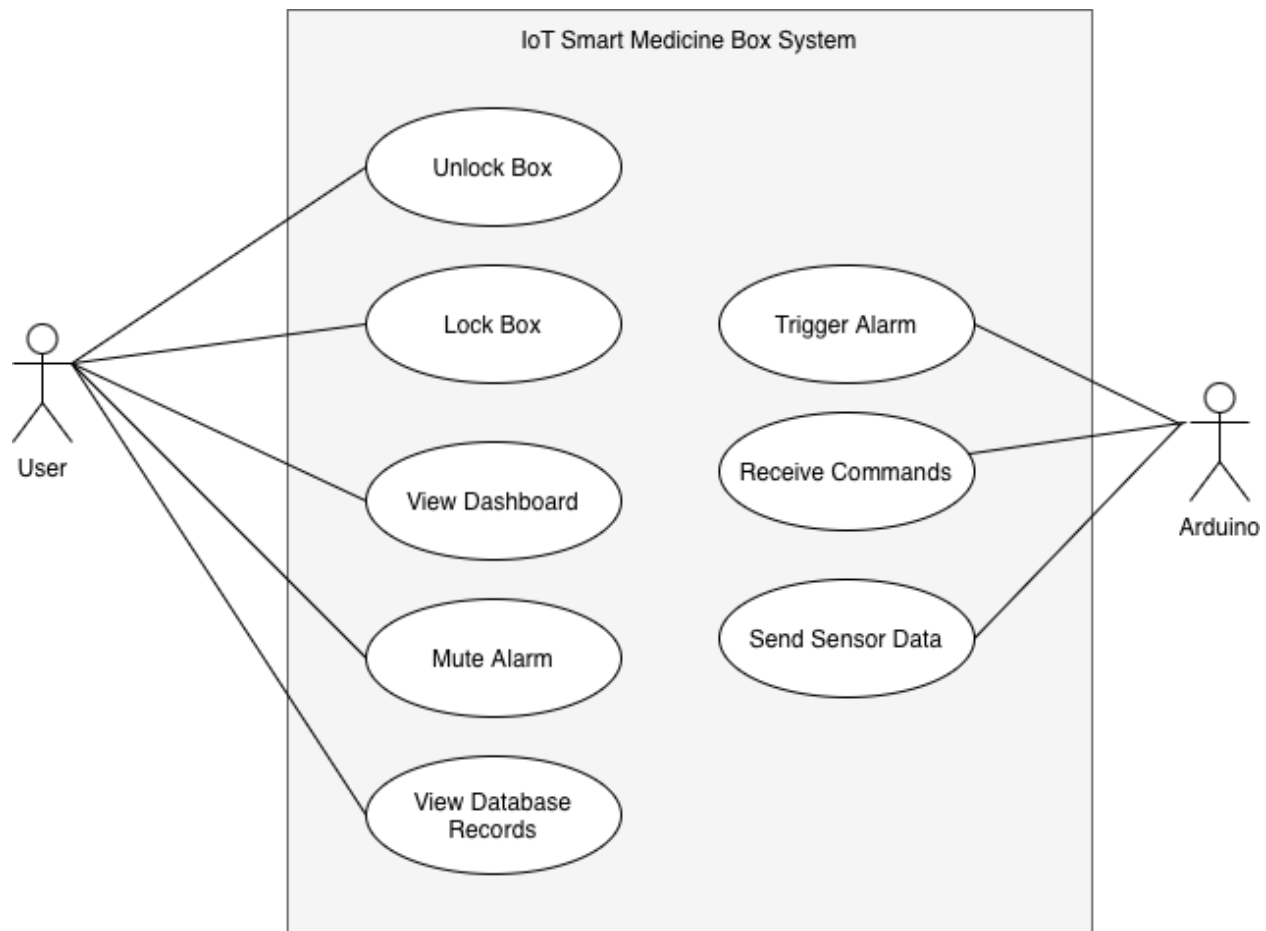
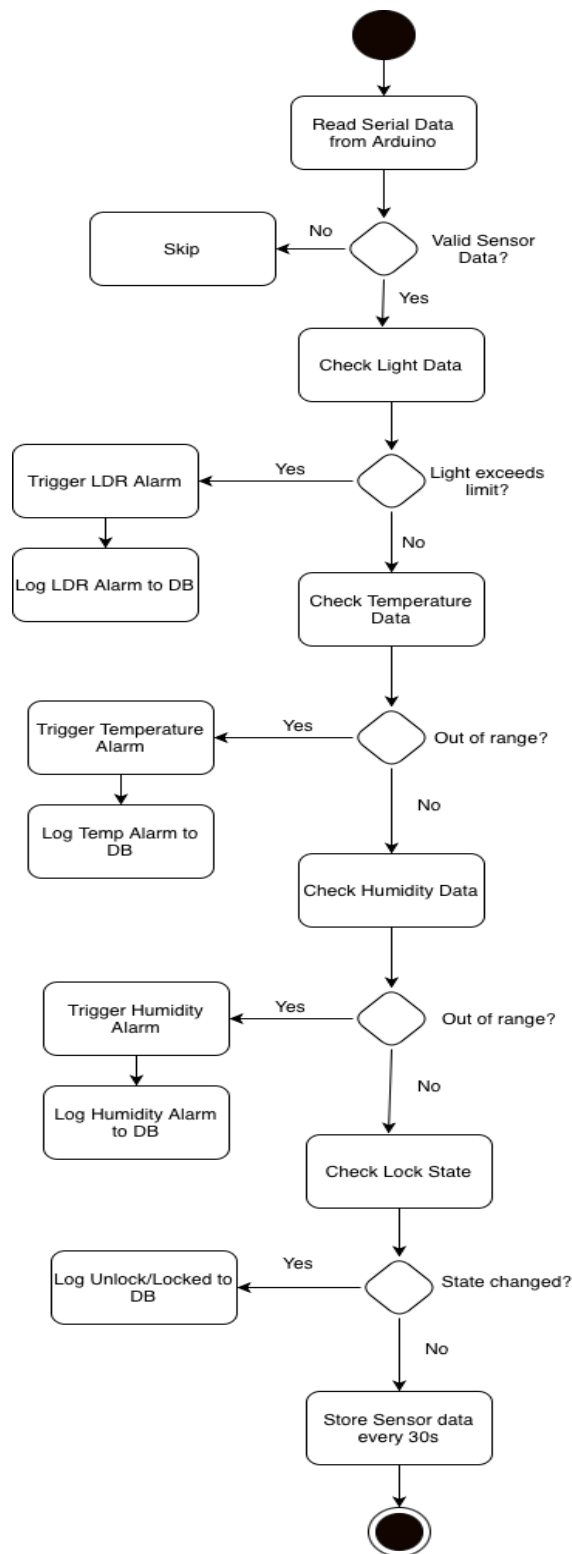


Figure 3. Use Case Diagram

The use-case diagram provides an overview of the system functions by identifying the User and Arduino's interactions with the smart box. The user can lock and unlock the box, view the dashboard, mute alarms and view database records. The Arduino sends data to the Edge Server, receives commands and triggers alarms.

3.2.2 Activity Diagram



This diagram captures the Edge Server's logic and analytics by showing the main software processes. It reads incoming data and validates before checking it. Each sensor input gets checked individually and if any condition is breached, the respective alarm is command gets set to the Arduino and logged. Any change in the lock state also gets logged to the access log table. Sensor data gets logged every 30 seconds by the Edge Server.

Figure 4. Activity Diagram

4 – Implementation

4.1 Sensors:

4.1.1 Analog

LDR - A Light Dependant Resistor will be used to measure the light inside of the box. The LDR's resistance changes based on the intensity of light hitting it. When the box is closed, the LDR will have high resistance, and when the box is opened the LDR will have low resistance. This is measured as an analog value that gets read by the Arduino. The value will get sent to the Edge Server which detects if it reaches past the light threshold while the box is still in a locked state. If so, the intrusion alarm gets triggered as the box has been opened while still "locked", meaning that it has been broken into. The LDR is important for detecting intrusions in the system and ensuring active security.

4.1.2 Digital

DHT11 - The DHT11 is a digital temperature and humidity sensor that has a capacitive humidity sensor, thermistor and a mini microcontroller inside of it. It will be used for measuring the moisture content in the box by detecting electrical resistance when the humidity changes. It will also measure the temperature inside of the box by measuring the changes in resistance due to surrounding temperature. The DHT11 communicates digitally through a single-wire digital protocol. The temperature and humidity values get sent to the Edge Server every second to detect if they are inside safe thresholds. If the temperature or humidity values are outside the safe thresholds, the temperature or humidity alarms will be triggered until they go back to a safe value. The DHT11 is important as certain medications are sensitive to heat and moisture.

4.2 Actuators

Servo Motor - The Servo Motor acts as the physical locking mechanism of the box. It can move between 0 and 180 degrees. When the box is locked, it rotates to 0 degrees, and when it is unlocked, it rotates to 90 degrees. It gets controlled by the Arduino when the correct password is entered through the keypad. The Edge Server can also control the Servo through serial communication.

RGB LED - The RGB LED works as a visual indicator for alarms. The RGB LED has three light-emitting diodes in a single component; green, blue, and red. The RGB LED flashes red for an intrusion alarm, blue for a temperature alarm, and green for a humidity alarm. It also flashes green when a correct password is entered, and red when the incorrect password is entered. It is controlled by using “digital Write()”, setting each pin to HIGH or LOW by the Arduino, when it receives the alarm command from the Edge Server. The RGB LED is important as it provides a visual status for the alarms and visual indication of which alarm is going off.

Buzzer - The buzzer acts as the audio for the alarms. A passive buzzer is used to emit different frequencies for each alarm. It gets controlled by the Arduino, using tone() when the Edge Server send an alarm command. It can also be controlled by a physical button to mute it. The buzzer is important as it provides an auditory indicator for alarms.

4.3 Input Devices

Keypad - A 4x4 matrix keypad is used for unlocking and locking the box. It uses a matrix scanning technique, mapping row and column pin configurations to each respective character. When the “#” is entered at the end, the input gets compared to the four-digit password and toggles the lock/unlocked state. The keypad is also used to stop the intrusion alarm.

Push Button - The push button acts as a mute control for the buzzer. When it is pressed, a “MUTE” command signal gets sent to the Arduino, silencing the buzzer when an alarm is going off without turning the alarm off. The mute button is useful for acknowledging any alarms since the temperature and humidity alarms only turn off when they go back to their safe zones.

4.4 Software/Libraries

4.4.1 Arduino Libraries

- **Keypad.h** - Handles the keyboard's input by mapping key presses to characters.
- **Servo.h** - Controls the servo position for the lock mechanism.
- **DHT.h** - Reads temperature and humidity data from the DHT11 sensor.

4.4.2 Python Libraries-

- **pymysql** - Used for connecting to the MySQL database and sending queries to it.
- **flask** - Used for hosting the web server.
- **pyserial** - Handles serial communication between Mac and Arduino over USB.

4.4.3 System Software -

- **MySQL**- Used to manage database for sensor data, alarm logs, access logs, and commands.
- **Arduino IDE** - Used for writing and uploading sketches to Arduino Uno.
- **Python 3** - Used for running python scripts for the edger server and flask.

Resources

Online Tutorials -

Arduino Lock Box. (2019). Projecthub.arduino.cc.

<https://projecthub.arduino.cc/LoganSpace42/arduino-lock-box-ff3d3d>

abales12. (n.d.). *LockBox.* Instructables. <https://www.instructables.com/LockBox/>

Instructables. (2020, March 16). *DHT11 Temperature & Humidity Sensor With Arduino.* Instructables. <https://www.instructables.com/DHT11-Temperature-Humidity-Sensor-With-Arduino/>

LeMaster Tech. (2023, November 30). *How to Connect and Control an Arduino From Python!* YouTube. <https://www.youtube.com/watch?v=UeybhVFqoeg>

Interfacing Arduino uno with LDR. (2021. June, 16). Projecthub.arduino.cc.

<https://projecthub.arduino.cc/electronicxfan123/interfacing-arduino-uno-with-ldr-61f455>

Guides-

Keypad data entry. A beginners guide. (2020, June 12). Arduino Forum.

<https://forum.arduino.cc/t/keypad-data-entry-a-beginners-guide/660309/2>